

Blind-Touch: Privacy-Preserving Fingerprint Authentication

Reproduction and local deployment of a homomorphic-encryption-based fingerprint authentication system (AAAI 2024)

DATE	11 June 2026
DEPLOYMENT	Single-host, three-container Docker (CPU-only)
STATUS	Complete — full pipeline verified end-to-end

Executive Summary

Blind-Touch performs fingerprint authentication **directly over homomorphically encrypted data**, so **the matching server never sees a fingerprint in plaintext**. If the server is breached, no usable biometric is exposed. This report documents a complete, independent reproduction of the system, rebuilt from the authors' source code as a three-container Docker deployment and exercised across its full lifecycle: dataset preparation, model training, user enrollment, encrypted server-side matching, and authentication.

The reproduction succeeded. A convolutional Siamese network trained for 150 epochs on the SOCOFing fingerprint dataset reached **99.58% validation accuracy**. A genuine authentication query was then matched entirely under encryption: the **correct identity scored 0.9984, exceeding the 0.99 acceptance threshold** and standing as the unambiguous top match, while the nearest non-matching identity scored 0.937 — a clear separation. The homomorphic matching step completed in approximately **0.23 seconds**.

Beyond reproducing the published result, several defects in the upstream code and documentation were identified and corrected (missing dataset preprocessing, an absent key-generation step, incompatible dependency versions, and uncreated runtime directories), and the build was made fully reproducible. Recommended next steps — a quantitative accept/reject evaluation and a multi-cluster deployment — are documented.

Validation accuracy	Genuine-match score	Threshold	HE match time	Decision
99.58%	0.9984	0.99	~0.23 s	Authenticated

1. Background and Objective

Conventional biometric authentication requires the server to store and compare raw biometric templates. If that server is breached, the leaked fingerprints cannot be "reset" like a password — they are compromised permanently. **Blind-Touch** addresses this using **homomorphic encryption (HE)**: a cryptographic scheme that allows mathematical operations to be performed on encrypted values, producing an encrypted result that, once decrypted, equals the result of the same operations on the original data. The authentication server therefore computes match scores on ciphertext it can never read.

Objective of this work: to independently reproduce the Blind-Touch system end-to-end on local hardware, validate that encrypted authentication produces correct accept/reject decisions, and document the process for the project team.

2. System Architecture

The deployment uses **three Docker containers** sharing a single Docker volume that stands in for the paper's networked storage. All inter-service communication is over HTTP.

Container	Role	Key software
client	User device. Runs the deep-learning model, generates encryption keys, encrypts fingerprint features, and issues authentication requests. Hosts a Jupyter environment.	TensorFlow 2.11, Microsoft SEAL, OpenCV
main	Authentication coordinator. Receives the encrypted query, forwards it to the cluster worker(s), aggregates the encrypted result, and returns it.	Flask, Microsoft SEAL
cluster1	Encrypted-matching worker. Holds the encrypted enrolled templates and performs the homomorphic match computation.	Flask, Microsoft SEAL

Shared volume layout (/workspace/shared_data inside every container):

- keys/ — public, secret, Galois, and relinearization keys
- models/ — the trained feature_model and matching model (TensorFlow SavedModel format)
- data/ — preprocessed fingerprint dataset and held-out test set
- ciphertexts/ — the enrolled encrypted templates and the encrypted query
- results/ — intermediate and final encrypted match outputs

Homomorphic encryption scheme. Microsoft SEAL's **CKKS** scheme (which supports approximate arithmetic on real numbers) was used with the following parameters:

- Polynomial modulus degree: **16384**
- Coefficient modulus chain: **[60, 40, 40, 40, 60]** bits
- Global scale: **2^{40}**
- Available SIMD slots per ciphertext: **8192**

The 8192 slots are central to the design: many enrolled identities are packed into a single ciphertext, so one homomorphic operation matches a query against all of them simultaneously.

Note on scope: the original paper distributes the enrolled database across multiple cluster servers. For this single-host reproduction we used **one cluster** (plus the coordinator), which exercises the identical cryptographic and matching logic at a scale appropriate to the available hardware. Extending to multiple clusters is configuration-only and is noted in Section 9.

3. The Machine-Learning Model

Authentication is framed as a **similarity-matching** problem using a **Siamese neural network** (twin networks with shared weights that learn whether two inputs are the same identity).

Feature extractor (applied identically to both inputs): five convolutional blocks with increasing depth — $32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512$ filters. Each block is a 3×3 convolution, batch normalization, a *swish* activation, and 2×2 max-pooling. A 224×224 grayscale fingerprint is reduced to a $7 \times 7 \times 512$ feature map, flattened to a **25 088-dimensional feature vector**, then unit-normalized.

Matching head: the two feature vectors are subtracted, passed through a 16-unit dense layer, squared, and fed to a final single-unit sigmoid layer that outputs a **match probability in [0, 1]**.

The model is trained with binary cross-entropy (genuine pair vs. impostor pair) using the Adam optimizer, and tracks accuracy, precision, recall, and F1.

Why this design matters for HE: the expensive, privacy-sensitive part of the matching head — the linear projection over the high-dimensional feature vector — is exactly what is executed homomorphically on the server. The non-linear final steps are applied by the client after decryption.

4. The Encrypted Authentication Protocol

1. **Enrollment (client, one-time per user).** The client runs each enrolled fingerprint through the feature extractor, encrypts the resulting feature vectors under CKKS, and stores the encrypted templates on the shared volume. Encryption keys are generated here; the **secret key never leaves the client**.
2. **Query (client).** At authentication time the client extracts and encrypts the feature vector of the presented fingerprint and sends this ciphertext to the `main` coordinator.
3. **Encrypted matching (server).** `main` forwards the encrypted query to `cluster1`, which homomorphically computes the matching head's linear projection between the query and **all enrolled templates at once** (using the packed-slot representation), producing an encrypted score vector. `main` aggregates and returns it.
4. **Decision (client).** The client decrypts the returned ciphertext, applies the final non-linear steps, and obtains a match probability per enrolled identity. If any score exceeds the **acceptance threshold of 0.99**, the user is **Authenticated**; otherwise **Rejected**.

Throughout, the server operates only on ciphertext and key material that cannot decrypt it.

5. Implementation Environment

- **Host OS:** Fedora 44, Linux kernel 7.0.11
- **Hardware:** CPU-only execution; 14 GB RAM + 8 GB swap
- **Containerization:** Docker (Compose), four images built on a shared base image

- **Base image contents:** Ubuntu 20.04, Python 3.8, TensorFlow 2.11, Microsoft SEAL (built from source), OpenCV (headless), Flask, aiohttp
- **Dataset: SOCOFing** (Sokoto Coin Fingerprint) — 6 000 real fingerprint images

All results in this report were produced on CPU.

6. Implementation Pipeline

The system was built and exercised in the following stages:

1. **Image build.** A shared base image was built with all native dependencies (notably SEAL, compiled from source), followed by three service images.
2. **Dataset preprocessing.** A purpose-written script converted the 6 000 raw SOCOFing `.BMP` images into a single normalized array of shape (6000, 224, 224, 1). *(This step was absent from the upstream materials — see Section 8.)*
3. **Model training.** The Siamese network was trained for **150 epochs** on CPU, executed headlessly. Trained weights were saved to the shared volume.
4. **Enrollment.** Encryption keys were generated and enrolled templates were encrypted and stored.
5. **Server start-up.** The `main` and `cluster1` services were started; both loaded the model and keys and initialized the CKKS context (confirming 8192 slots).
6. **Authentication.** An encrypted query for an enrolled identity was submitted and the accept/reject decision was verified.

All long-running and server steps were run **non-interactively** (headless notebook execution) for reproducibility.

7. Results

7.1 Model training

- **Epochs completed:** 150 / 150 (CPU)
- **Final validation accuracy:** 0.9958
- **Final validation loss:** 0.0186
- **Throughput:** $\approx$ 124 seconds/epoch; $\approx$ 7.6 hours wall-clock total
- Validation accuracy crossed 0.96 within the first six epochs and converged to $\sim$ 0.99, indicating a well-trained, stable model.

7.2 Encrypted authentication

A genuine query (enrolled identity #467) was authenticated end-to-end under encryption:

Metric	Value
Match score for the correct identity	0.998441
Highest score across all enrolled identities	0.998441 (the correct identity)
Next-highest (impostor) score	0.937245
Acceptance threshold	0.99
Identities above threshold	exactly one — the correct identity
Decision	Authenticated
Homomorphic matching time	≈ 0.23 s

The genuine identity was both above threshold and the unambiguous top match, with a clear margin over the nearest impostor. This confirms the reproduced system performs **correct, privacy-preserving authentication**.

Evidence

```

● ● ● Training — final epochs (CPU, 150/150)

$ tail -n 6 train_progress.log # final epochs of the 150-epoch CPU run
[TimeBudget] epoch 145/150 in 124.0s | elapsed 444.0min | avg 183.7s/ep | loss=0.0142
val_loss=0.0095 val_acc=0.9975
[TimeBudget] epoch 146/150 in 123.9s | elapsed 446.1min | avg 183.3s/ep | loss=0.0232
val_loss=0.0251 val_acc=0.9917
[TimeBudget] epoch 147/150 in 123.8s | elapsed 448.1min | avg 182.8s/ep | loss=0.0127
val_loss=0.0243 val_acc=0.9908
[TimeBudget] epoch 148/150 in 123.9s | elapsed 450.2min | avg 182.5s/ep | loss=0.0154
val_loss=0.0227 val_acc=0.9925
[TimeBudget] epoch 149/150 in 123.7s | elapsed 452.3min | avg 182.1s/ep | loss=0.0183
val_loss=0.0201 val_acc=0.9942
[TimeBudget] epoch 150/150 in 123.8s | elapsed 454.3min | avg 181.7s/ep | loss=0.0186
val_loss=0.0186 val_acc=0.9958

```

Figure 1. Final epochs of the 150-epoch CPU training run (validation accuracy 0.9958).

```

● ● ● Encrypted authentication — result

$ jupyter nbconvert --execute Blind-Touch-Auth.ipynb # encrypted authentication

Number of slots: 8192
Keys loaded from /workspace/shared_data/keys
weight shape: (25088, 16) | bias shape: (16,)
query idx : 467
1/1 [=====] - ETA: 0s
1/1 [=====] - 0s 84ms/step
TOTAL TIME (s): 0.22671294212341309
self-match score (enrolled idx 467): 0.998441
max score over enrolled set : 0.998441 (idx 467)
top-5 enrolled scores : [(467, 0.998441), (277, 0.937245), (155, 0.535795), (392,
0.023805), (430, 0.014617)]
indices over THRESHOLD 0.99 : [467]
=====
RESULT: Authenticated
=====

```

Figure 2. End-to-end encrypted authentication: identity #467 accepted (score 0.9984 > 0.99 threshold).

```
Services running + CKKS context

$ docker compose ps
NAME                STATUS    PORTS
blindtouch-client    Up 9 hours  0.0.0.0:8888->8888/tcp, [::]:8888->8888/tcp
blindtouch-cluster1  Up 2 hours  8090/tcp
blindtouch-main       Up 2 hours  0.0.0.0:8090->8090/tcp, [::]:8090->8090/tcp

$ docker compose logs cluster1 | grep "Number of slots"
blindtouch-cluster1 | Number of slots: 8192
blindtouch-main     | Number of slots: 8192
```

Figure 3. The three services running, with both HE servers reporting 8192 CKKS slots.

8. Deviations from the Upstream Materials and Defects Corrected

The published code and accompanying guide required several corrections before the system would run. Documenting these is a substantive part of this work.

Defects fixed in the build:

- **Conflicting deep-learning dependencies.** The dependency list pinned two mutually incompatible library versions; resolved by standardizing on the versions bundled with TensorFlow 2.11.
- **Missing server dependencies.** The web-server libraries (Flask, aiohttp) were required by the code but absent from the dependency list; added.
- **OpenCV conflict.** A transitive dependency pulled in an incompatible OpenCV build that broke image loading; pinned to the correct headless build.

Defects fixed in the application code/notebooks:

- **No dataset preprocessing existed.** The training code expected a preprocessed array that neither the repository nor the guide produced; a preprocessing script was written.
- **No key-generation step existed.** The client code only *loaded* encryption keys but never generated them; a key-generation step was added.
- **Unconfigured file paths.** Numerous storage paths were left as placeholders in the source and were filled in to use the shared volume.
- **Missing runtime directories.** Two output directories (`ciphertexts/` and `results/`) were never created by the code. Without them, the encryption library failed with a low-level I/O error that surfaced, misleadingly, as a "version incompatibility" on the client. Both directories are now created before use.

Documentation correction: - The guide assumed the server was a notebook watching a shared folder. In reality the server is a set of Flask web applications communicating over HTTP. The deployment topology was corrected accordingly.

9. Limitations and Future Work

- **Single cluster.** One matching worker was used; the paper's multi-cluster sharding is a configuration extension and would demonstrate horizontal scaling.
- **Training time.** A full 150-epoch training run took ~7.6 hours on CPU; this is the principal time cost of a fresh end-to-end reproduction and can be shortened by reducing the epoch count for demonstration purposes.
- **Single dataset / threshold.** Results use the SOCOFing dataset and the upstream acceptance threshold of 0.99. A formal evaluation (false-accept / false-reject rates across many trials, including known impostors and non-enrolled queries) would quantify operating-point accuracy.

- **Development-grade serving.** The services run on Flask's development server, appropriate for a demonstration but not production.

Recommended next steps: (1) run a quantitative accept/reject evaluation including non-enrolled queries; (2) scale to multiple clusters to mirror the paper's distributed design.

10. Reproducibility

The deployment is fully scripted. The complete, exact command sequence — including the two directory-creation fixes and the headless execution commands for training, enrollment, and authentication — is recorded in the project's `RESUME.md`, and every deviation from the upstream materials is recorded in `SETUP-NOTES.md`. A fresh reproduction follows: build images → preprocess dataset → train → enroll → start servers → authenticate.

11. Conclusion

The Blind-Touch privacy-preserving fingerprint authentication system was successfully reproduced and verified end-to-end on local CPU hardware. A high-accuracy matching model (99.58 % validation accuracy) was trained, and a genuine fingerprint was authenticated **entirely over encrypted data** — the matching server at no point had access to the plaintext biometric — yielding a correct **Authenticated** decision with a clear margin over impostor scores and sub-second homomorphic matching. In the course of the work, multiple defects in the upstream code and documentation were identified and fixed, and the build was made reproducible. The result is a working, well-documented demonstration of homomorphic-encryption-based biometric authentication and a solid foundation for the quantitative evaluation and multi-cluster extensions outlined above.